

选课管理系统 - 项目开发方案 - 第二阶段

学号	姓名	项目组角色
211850016	李薛成	Scrum master、开发团队人员
211250234	张铭铭	产品负责人、开发团队人员
211250236	陈皓鑫	开发团队人员
211250245	张哲恺	开发团队人员

Note

与上次作业的差异：

- 1. 更正了需求验证与需求确认的内容与顺序。
- 2. 优化了 WBS 分解，改进了系统测试部分的分解内容。
- 3. 重新进行规模相对大小矩阵估算和资源相对大小矩阵估算，并依此重新修改日程计划中的相关计算。
- 4. 修改了日程计划中的时间安排，使其更加合理。
- 5. 使用燃尽图替代里程碑进行进度管理。
- 6. 在质量计划中对 Yield、A/FR、PQI、Review Rate 四个质量指标进行了详细的讨论。
- 7. 使用 PSP 质量策略对质量计划进行完善。
- 8. 重新对风险评估进行了讨论，并将机遇也纳入风险评估的讨论范围内。

在答辩后做出的改进：

- 1. 删除了资源相对大小矩阵，避免 PROBE 代理识别产生冲突与冗余。
- 2. 在规模相对大小矩阵中增加了以文档页数为单位的需求文档、设计文档的规模相对大小估算。
- 3. 删除了规模相对大小矩阵中有关评审的规模估算。
- 4. 强调了若因项目技术难度较大，导致开发团队难以按时完成开发任务，按照任务优先级进行功能调整。
- 5. 删除了质量计划中所谓的 PSP 成熟度级别，但保留了原有的质量保障措施。。
- 6. 优化了 A/FR 中关于失效成本的描述，避免编译时间产生歧义。

选课管理系统 - 项目开发方案 - 第二阶段

产品目标和业务目标

产品目标

业务目标

角色分配和小组目标定义

角色分配

小组目标

开发流程定义与策略选择

开发流程定义

开发策略

Sprint 目标

开发框架

软硬件设备

整体计划

概要产品需求

产品概要设计 & PROBE 估算

- WBS 分解
- 估算 & 日程计划
- 任务优先级
- 整体日程计划
- 燃尽图项目跟踪管理
- 质量计划
 - 过程记录和分析
 - 编码规范
 - 单元测试
 - 集成测试
 - 模块通用要求
 - 用户界面模块要求
 - 数据持久化模块要求
 - 功能、性能、安全性评估
 - 制定个人目标和计划
 - 过程改进和学习
 - 需求确认和验证
 - 代码评审
 - 软件设计
 - 潜在冲突协调 - 质量保障活动对正常开发活动的影响
 - 冲突问题
 - 解决方案
 - 质量监控指标
 - Yield
 - A/FR
 - PQI
 - Review Rate
- 个人计划和计划平衡
- 风险评估
 - 风险识别
 - 风险应对
 - 机遇识别和应对

产品目标和业务目标

产品目标

- 提供一个基于 WEB 的在线选课系统，需要保证系统稳定性，能够应对至少 1000 人同时参加选课的情况。
- 该选课系统需要支持邮件发送功能，可以发送选课通知、选课结果等信息至学生邮箱，同时也支持用户注册、密码找回等信息的邮件发送。
- 该选课系统以院系为单位，支持课程的导入、保存和修改，同时也支持对课程进行设置，包括设定选课人数上限、先修课程限制、选课者年级限制等。
- 该选课系统提供 LLM 选课助手功能，支持用户用自然语言交互，用户可以了解感兴趣的信息，例如，课程内容、要求等各类信息。
- 该选课系统需要设定选课期限，在期限内可以选课，超出期限则不能选课。
- 该选课系统需要对特殊情况的学生，例如，重修、辅修等进行处理。
- 该选课系统需要支持管理员的以自然语言交互方式的指令操作，允许管理员用自然语言完成如将某一位同学加入某个班（课程）、修改某一门课程的设置好的信息等操作。

业务目标

- 质量：
 - 确保选课流程和规定符合学校制定的相关规则和政策。
 - 确保学生选课的高效性和便捷性，以及管理员管理的便捷性。
 - 系统稳定可靠，具备较强的鲁棒性，并能够快速响应，提供良好的用户体验。
- 成本：尽量减少开发成本，提高开发效率，将开发成本控制在合理范围内。
- 工期：该系统需要在项目方案确定后的 14 天内交付。

角色分配和小组目标定义

角色分配

角色：

- Scrum master：Scrum master 需要促进和支持 Scrum，帮助团队每个人理解 Scrum 理论、实践、规则和价 值，负责组织各种 Scrum 会议，例如 Sprint 计划会议、每日 Scrum 会议、评审会议和回顾会议，并在团队出 现问题时能起到协调作用，帮助团队解决问题。另外，Scrum master 需要积极提出过程优化建议，提升 Scrum 团队生产率。
- 产品负责人：产品负责人的职责就是将开发团队开发的产品价值最大化。产品负责人需要负责管理产品待办列 表，确保待办列表中的工作项按照优先级顺序排列，以便开发团队能够优先处理最重要的工作项。产品负责人还 需要与开发团队密切沟通，确保开发团队理解产品待办列表中的工作项，使得开发团队能够按照产品负责人的期 望完成工作。
- 开发团队：开发团队需要完成产品负责人指定的工作项，确保在每个 Sprint 结束时交付可用的软件。开发团队 需要保持高效的工作状态，确保在每个 Sprint 结束时交付的软件质量高、功能完整。

角色分配如下：

姓名	项目组角色
李薛成	Scrum master、开发团队人员
张铭铭	产品负责人、开发团队人员
陈皓鑫	开发团队人员
张哲恺	开发团队人员

小组目标

功能目标：

- 在 14 天内交付一个支持至少 1000 人同时参加选课的在线选课系统，实现需求文档中的全部核心功能。

质量目标：

- 系统稳定可靠，具备较强的鲁棒性，并能够快速响应，在 1000 人的并发量级下响应时间控制在 1s 内，提供良 好的用户体验。
- 核心业务流程的测试用例覆盖率达到 90% 以上，核心功能模块的代码覆盖率达到 98% 以上。
- 代码复杂度控制在合理范围内，代码质量达到较高水平，具有较好的可拓展性，符合 Google 制定的编码规范。

流程目标：

- 每个 Sprint 周期为 4 天，每个 Sprint 周期完成至少 3 个用户故事。
- 按 Scrum 流程执行，记录全过程并存档。

协作目标：

- 团队成员尽职尽责，完成自己职责内的工作。
- 团队成员之前相互协作，及时沟通，积极协调配合。

开发流程定义与策略选择

开发流程定义

由于选课管理系统的开发周期较短，团队选择采用 Scrum 方法来完成在线选课系统的开发。将开发周期划分为 3 个 Sprint，每个 Sprint 的周期为 4 天。在每个 Sprint 中，团队会完成一部分功能的开发，并在所有开发流程结束后交付可用的软件，进行用户培训与仿真模拟 / 用户验收测试。

团队的开发流程如下：

- 需求开发阶段：该阶段团队对选课管理系统的需求进行分析，确定系统的功能和性能需求，确保符合使用预期。需求开发阶段活动包括需求获取、需求分析、需求确认，最终的产物是需求规格说明书和产品待办列表。
- 系统开发阶段：该阶段可以分为框架开发和功能开发两个子阶段。框架开发阶段主要完成选课管理系统的基础框架搭建，包括数据库设计、系统架构设计等；功能开发阶段按照需求和产品待办列表，完成选课管理系统的具体功能开发。
- 系统测试阶段：该阶段团队对选课管理系统进行测试，包括测试开发、单元测试、集成测试和用户验收测试。测试阶段的目的是确保选课管理系统的质量，保证系统的可用性、稳定性和可靠性。

关于开发中的 Sprint，团队约定如下：

- 每个 Sprint 的周期为 4 天，团队每天下班前进行15分钟站立会议，进行报告和协调
- Sprint 计划会议
 - 每个 Sprint 开始时，召开 Sprint 计划会议。
 - 在 Sprint 计划会议上，团队会确定本 Sprint 要完成的工作项，并评估优先级、风险和复杂度，将工作项添加到产品待办列表中。
 - 会根据产品待办列表中的工作项，制定本 Sprint 的开发计划。
 - 制定 Sprint 目标和交付成果。
- Sprint 评审会议
 - 每个 Sprint 结束时，召开 Sprint 评审会议，自顶向下地进行代码评审、需求验证、文档验证。
 - 在 Sprint 评审会议上，开发团队会展示本 Sprint 完成的工作项，讨论工作项的完成情况，接受产品负责人的反馈。
 - 会根据产品负责人的反馈，调整产品待办列表中的工作项，为下一个 Sprint 制定开发计划。
- Sprint 回顾会议
 - 每个 Sprint 评审会议结束后，召开 Sprint 回顾会议。
 - 在 Sprint 回顾会议上，团队会讨论本 Sprint 的工作情况，总结本 Sprint 的经验教训，提出改进意见。
 - 会根据 Sprint 回顾会议的结果，优化团队的工作流程，提升团队的工作效率。
- 产品增量交付
 - 每个 Sprint 结束时，团队会交付一个可用的产品增量，确保产品质量和功能完整。

开发策略

Sprint 目标

团队使用 Scrum 方法进行开发，将开发周期划分为 3 个 Sprint，每个 Sprint 的周期为 4 天。在每个 Sprint 中，团队会完成一部分功能的开发，并在所有开发流程结束后交付可用的软件，进行用户培训与仿真模拟 / 用户验收测试。

- Sprint 1
 - 完成选课管理系统的需求分析，确定系统的功能和性能需求，产出需求规格说明书和和产品待办列表。
 - 完成选课系统的系统架构设计、数据库设计。
 - 完成 DevOps 环境的搭建，包括代码仓库、持续集成、持续部署、服务监视等。
 - 完成用户登入、注册、密码找回等功能与用户权限设计。
 - 开发测试用例，进行单元测试与已完成系统的集成测试。
- Sprint 2
 - 开发选课管理的基础功能，包括选课相关功能和管理功能。
 - 开发选课管理系统的邮件发送功能。
 - 开发基础的基于 LLM 的自然语言交互接口。
 - 开发测试用例，进行单元测试与已完成系统的集成测试。
- Sprint 3
 - 完善基于 LLM 的自然语言交互功能，开发选课助手功能与管理员自然语言交互功能。
 - 完成测试开发后，对整个系统进行单元测试、集成测试（包含系统测试、安全性测试、压力测试等）。
 - 根据评审与测试结果，进行代码重构与优化，对系统的性能和安全性进行优化。
- 完成 Sprint 之后
 - 进行用户培训，确保用户（管理员、学生）能够熟练使用选课管理系统。
 - 进行仿真模拟 / 用户验收测试，确保选课管理系统的质量和性能。
 - 交付选课管理系统。

开发框架

项目基本上使用现有的主流开发框架和现成的组件，经过团队成员会议讨论，根据成员对不同框架的熟悉程度和项目需求，最终确定使用以下框架：

- 客户端：使用 React 框架进行客户端开发，React 是一个用于构建用户界面的 JavaScript 库，可以帮助开发者构建大型 Web 应用，提供了丰富的组件和工具，可以快速搭建客户端应用。
- 服务端：使用基于 Go 语言的 Gin 框架进行服务端开发，Gin 是一个用于构建 Web 应用的 Go 语言框架，提供了丰富的中间件和工具，可以快速搭建服务端应用，对高并发有较好的支持。
- 数据库：使用 PostgreSQL 数据库进行数据存储，PostgreSQL 是一个开源的关系型数据库管理系统，支持 SQL 语言，提供了丰富的数据类型和功能，可以满足选课管理系统的并发需求与结构化数据存储需求。
- 邮件组件：使用 Go 语言的 mail 库进行邮件发送功能的开发，mail 是一个用于发送邮件的 Go 语言库，提供了丰富的邮件发送功能，可以满足选课管理系统的邮件发送需求。
- Excel 组件：使用 Go 语言的 excelize 库进行 Excel 文件导入导出功能的开发，excelize 是一个用于读写 Excel 文件的 Go 语言库，提供了丰富的 Excel 文件读写功能，可以满足选课管理系统的 Excel 文件导入导出需求。
- 大语言模型组件：使用由另一团队 Finetune 后得到的 Llama 3 大语言模型进行自然语言交互功能的开发，Meta Llama 3 是一个开源的大语言模型，性能优秀，可以根据不同的需求进行微调，拥有 Function Call（函数调用）功能，可以满足选课管理系统的自然语言交互需求。

软硬件设备

- 部署服务器
 - 硬件支持
 - CPU: 22 核 3.6 GHz 及以上 (如 Intel Xeon E5-2699 v4)
 - 内存: 64 GB 及以上
 - 存储: 1 TB RAID 1 SSD
 - 网络带宽: 校内 1 Gbps
 - 软件支持
 - 操作系统: Ubuntu 20.04 LTS
 - 容器引擎: Docker
 - 数据库: PostgreSQL
 - 服务端环境: Go 1.16
 - 客户端环境: Node.js 14.17
 - WEB 服务器: Nginx
 - 防火墙: UFW
- 开发环境
 - 硬件支持
 - 四台开发机 (由团队成员自行准备)
 - 软件支持
 - 操作系统: Windows 11
 - 开发工具: WebStorm (客户端开发)、GoLand (服务端开发)
 - 测试工具: Testify (Go 单元测试)、Apache Bench (压力测试)
- DevOps 环境
 - 代码仓库: GitLab
 - CI/CD: GitLab CI/CD
 - 监视工具: Prometheus、Grafana

整体计划

概要产品需求

- 系统基本特性
 - 平台要求: 基于 WEB, 稳定, 快速, 支持至少 1000 人同时参加选课。
 - 邮件支持: 支持邮件发送功能, 包括选课通知、选课结果、用户注册、密码找回等功能。
- 基础通用功能
 - 基础通用功能主要包括用户注册、登录、数据库交互、用户信息管理等基础通用功能。
- 课程管理功能
 - 课程导入: 支持管理员导入、保存和修改课程信息。导入格式由可下载的 Excel 文件定义。
 - 课程信息设置: 支持管理员设置课程信息。包括选课人数上限、先修课程、年级限制、课程性质和课程简介等。

- 选课助手功能
 - 安全性：选课助手对于提供的信息需要保守，对于不确定或者系统没有的信息，选课助手需要明确告知用户不确定或者没有相关信息。
- 选课限制功能
 - 可选课程限制：系统支持对每个学生每学期最多选课数目进行上限限制。
 - 必修课程限制：系统支持对每个学生必修课程的选课进行限制。学生必须选修当季开设的必修课程，不允许自行退课。
 - 选课时间限制：学生只能在系统设定的选课时间内进行选课操作，超出时间将无法选课。
- 课表生成功能
 - 课表生成：在选课结果确定后，系统根据课程信息自动生成学生的课表。
 - 课表导出：学生可以将生成的课表导出为 Excel 文件，方便学生查看。
- 特殊情况处理功能
 - 特殊情况学生处理：系统支持对特殊情况学生按照学校规定进行特殊处理。例如重修、留级、休学、辅修等。
- 管理员自然语言交互功能
 - 管理员自然语言交互：基于 LLM，允许管理员使用自然语言进行课程信息修改、将一位学生加入某门课程等操作。

产品概要设计 & PROBE 估算

WBS 分解

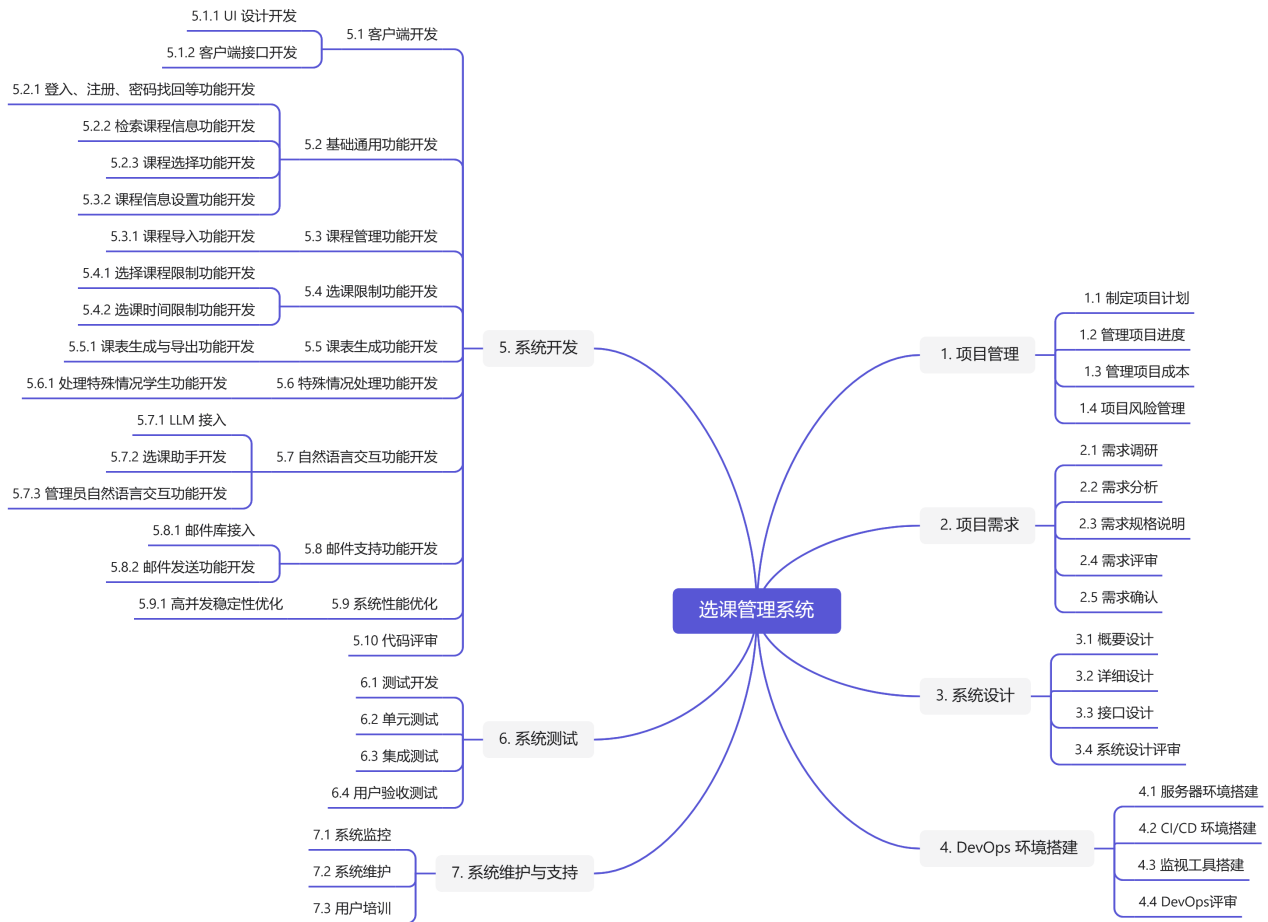
根据上述产品需求，我们将产品的工作分解如下：

- 1. 项目管理
 - 1.1 制定项目计划
 - 1.2 管理项目进度
 - 1.3 管理项目成本
 - 1.4 项目风险管理
- 2. 项目需求
 - 2.1 需求调研
 - 2.2 需求分析
 - 2.3 需求规格说明
 - 2.4 需求评审
 - 2.5 需求确认
- 3. 系统设计
 - 3.1 概要设计
 - 3.2 详细设计
 - 3.3 接口设计
 - 3.4 系统设计评审
- 4. DevOps 环境搭建
 - 4.1 服务器环境搭建

- 4.2 CI/CD 环境搭建
- 4.3 监视工具搭建
- 4.4 DevOps 评审
- 5. 系统开发
 - 5.1 客户端开发
 - 5.1.1 UI 设计开发
 - 5.1.2 客户端接口开发
 - 5.2 基础通用功能开发
 - 5.2.1 登入、注册、密码找回等功能开发
 - 5.2.2 检索课程信息功能开发
 - 5.2.3 课程选择功能开发
 - 5.3 课程管理功能开发
 - 5.3.1 课程导入功能开发
 - 5.3.2 课程信息设置功能开发
 - 5.4 选课限制功能开发
 - 5.4.1 选择课程限制功能开发
 - 5.4.2 选课时间限制功能开发
 - 5.5 课表生成功能开发
 - 5.5.1 课表生成与导出功能开发
 - 5.6 特殊情况处理功能开发
 - 5.6.1 处理特殊情况学生功能开发
 - 5.7 自然语言交互功能开发
 - 5.7.1 LLM 接入
 - 5.7.2 选课助手开发
 - 5.7.3 管理员自然语言交互功能开发
 - 5.8 邮件支持功能开发
 - 5.8.1 邮件库接入
 - 5.8.2 邮件发送功能开发
 - 5.9 系统性能优化
 - 5.9.1 高并发稳定性优化
 - 5.10 代码评审
- 6. 系统测试
 - 6.1 测试开发
 - 6.2 单元测试
 - 6.3 集成测试
 - 6.4 用户验收测试
- 7. 系统维护与支持
 - 7.1 系统监控

- 7.2 系统维护
- 7.3 用户培训

WBS 如下图所示：



估算 & 日程计划

编号	WBS 分解位置	工作名称	功能点大小	规模 (页数/LOC)	代码累计规模 (LOC)	需要时间资源 (人时)	累计时间资源 (人时)	完成时间 (Day X)	负责人
1	1.1	制定项目计划	S	-	-	4	4	1	李薛成
2	1.2	管理项目进度	S	-	-	4	8	14	李薛成
3	1.3	管理项目成本	VS	-	-	2	10	14	李薛成

编号	WBS分解位置	工作名称	功能点大小	规模 (页数/LOC)	代码累计规模 (LOC)	需要时间资源 (人时)	累计时间资源 (人时)	完成时间 (Day X)	负责人
4	1.4	项目风险管理	S	-	-	4	14	14	李薛成
5	2.1	需求调研	S	-	-	4	18	1	张铭铭
6	2.2	需求分析	S	2	-	4	22	1	张铭铭
7	2.3	需求规格说明	S	8	-	4	26	2	张铭铭
8	2.4	需求评审	M	-	-	8	34	2	全体
9	2.5	需求确认	VS	-	-	4	38	2	全体成员
10	3.1	概要设计	M	12	-	8	46	3	张铭铭
11	3.2	详细设计	M	24	-	4	50	3	张铭铭
12	3.3	接口设计	L	8	-	12	62	10	张铭铭
13	3.4	系统设计评审	S	-	-	12	74	10	全体
14	4.1	服务器环境搭建	VS	100	100	2	76	4	张哲恺
15	4.2	CI/CD环境搭建	S	100	200	2	78	4	张哲恺

编号	WBS分解位置	工作名称	功能点大小	规模 (页数/LOC)	代码累计规模 (LOC)	需要时间资源 (人时)	累计时间资源 (人时)	完成时间 (Day X)	负责人
16	4.3	监视工具搭建	M	200	400	4	82	4	张哲恺
17	4.4	DevOps评审	S	-	-	4	86	4	全体
18	5.1.1	UI设计开发	L	500	900	16	102	11	张铭铭
19	5.1.2	客户端接口开发	L	300	1200	12	114	11	张铭铭
20	5.2.1	登入、注册、密码找回等功能开发	M	150	1350	6	120	4	陈皓鑫
21	5.2.2	检索课程信息功能开发	S	70	1420	4	124	6	陈皓鑫
22	5.2.3	课程选择功能开发	S	70	1490	4	128	6	陈皓鑫
23	5.3.1	课程导入功能开发	VS	30	1520	2	130	7	陈皓鑫
24	5.3.2	课程信息设置功能开发	VS	30	1550	2	132	7	陈皓鑫
25	5.4.1	选择课程限制功能开发	VS	30	1580	2	134	8	陈皓鑫
26	5.4.2	选课时间限制功能开发	VS	30	1610	2	136	8	陈皓鑫
27	5.5.1	课表生成与导出功能开发	VS	30	1640	2	138	8	陈皓鑫

编号	WBS分解位置	工作名称	功能点大小	规模 (页数/LOC)	代码累计规模 (LOC)	需要时间资源 (人时)	累计时间资源 (人时)	完成时间 (Day X)	负责人
28	5.6.1	处理特殊情况学生功能开发	S	70	1710	4	142	8	陈皓鑫
29	5.7.1	LLM接入	M	150	1860	6	148	9	李薛成
30	5.7.2	选课助手开发	M	150	2010	6	154	10	李薛成
31	5.7.3	管理员自然语言交互功能开发	M	150	2160	6	160	11	李薛成
32	5.8.1	邮件库接入	VS	30	2190	2	162	7	张哲恺
33	5.8.2	邮件发送功能开发	VS	30	2220	2	164	7	张哲恺
34	5.9.1	高并发稳定性优化	M	150	2370	8	172	11	李薛成
35	5.1	代码评审	L	-	2370	36	208	11	全体
36	6.1	测试开发	M	100	2470	10	218	11	陈皓鑫
37	6.2	单元测试	L	150	2620	12	230	12	张铭铭
38	6.3	集成测试	L	500	3120	16	246	12	张铭铭
39	6.4	用户验收测试	S	-	-	4	250	14	全体
40	7.1	系统监控	S	-	-	4	254	14	张哲恺

编号	WBS分解位置	工作名称	功能点大小	规模 (页数/LOC)	代码累计规模 (LOC)	需要时间资源 (人时)	累计时间资源 (人时)	完成时间 (Day X)	负责人
41	7.2	系统维护	S	-	-	4	258	14	张哲恺
42	7.3	用户培训	S	-	-	12	270	14	全体

参考的规模相对大小矩阵（设计时规模单位为文档页数，编码时规模单位为LOC）如下：

类型	VS	S	M	L	VL
需求分析	1	2	5	8	10
需求规格说明文档	4	8	12	20	30
系统概要设计	4	8	12	20	30
系统详细设计	8	16	24	32	40
系统接口设计	1	2	4	8	10
DevOps 脚本开发	50	100	200	300	500
客户端 UI 开发	100	200	300	500	1000
客户端接口开发	50	100	200	300	500
服务端组件开发	30	70	150	300	500
测试用例开发	10	20	50	100	150
单元测试	20	50	100	150	200
集成测试	100	200	300	500	800

注：

- DevOps 脚本开发：指的是 CI/CD 脚本、监视工具脚本、服务器环境搭建脚本等脚本文件开发。
- 客户端 UI 开发：指的是客户端全部的 UI 设计开发，包括客户端页面设计、客户端交互设计等。
- 客户端接口开发：指的是客户端全部的接口开发，根据设计好的接口文档进行接口开发。
- 服务端组件开发：指的是服务端的每一个功能组件的开发，LOC 以 Go 语言为准。
- 在系统设计完之后客户端可与服务端并行开发，客户端开发并不需要等待服务端开发完成，不过客户端开发进程可与服务端开发进程同步。

任务优先级

团队对每个任务进行了优先级排序，以便更好地安排工作。优先级分为五个等级，从高到低由 5 至 1 分级。在开发过程中，团队将根据任务的优先级来安排工作，确保高优先级任务的及时完成。

若出现因项目技术难度较大，导致开发团队难以按时完成开发任务的情况，团队会保证优先级大于等于 4 的任务能够按时完成，优先级小于 4 的任务视高优先级任务完成情况与时间资源剩余而定。

编号	优先级	理由
1-22,34-42	5	这部分内容是系统的基础功能与底层设计，是整个系统的核心，必须优先完成。
23,24	5	课程管理功能是选课系统的核心功能，如果缺少这部分功能，选课系统将没有具体课程信息，无法进行选课。需要优先完成。
25,26	5	选课限制功能是选课系统的重要功能，如果缺少这部分功能，学生可以随意选课，将导致无法按照学校和院系的规定进行选课。需要优先完成。
27	3	课表生成与导出功能是方便学生可视化地查看自己选课情况，方便学生安排自己的选课计划。缺少了这部分功能，系统仍然可以正常运行。所以优先级较低。
28	2	特殊情况学生处理功能是对特殊情况学生进行特殊处理的功能，是系统的附加功能。由于特殊情况学生数量较少，缺少了这部分功能，特殊情况学生的选课可以由管理员手动处理。所以优先级最低。
29-31	3	自然语言交互功能是选课系统的附加功能，是提高用户体验的功能。缺少了这部分功能，用户可以通过其他方式进行选课，管理员可以通过手动操作进行管理。同时 LLM 效果和稳定性未知，所以优先级较低。
32,33	4	邮件支持功能主要是通知学生选课结果和注册、找回密码。缺少了这部分功能，学生可以通过其他方式获取选课结果，通过管理员手动操作进行注册和找回密码。同时该功能规模与耗时较小，可以优先完成。

整体日程计划

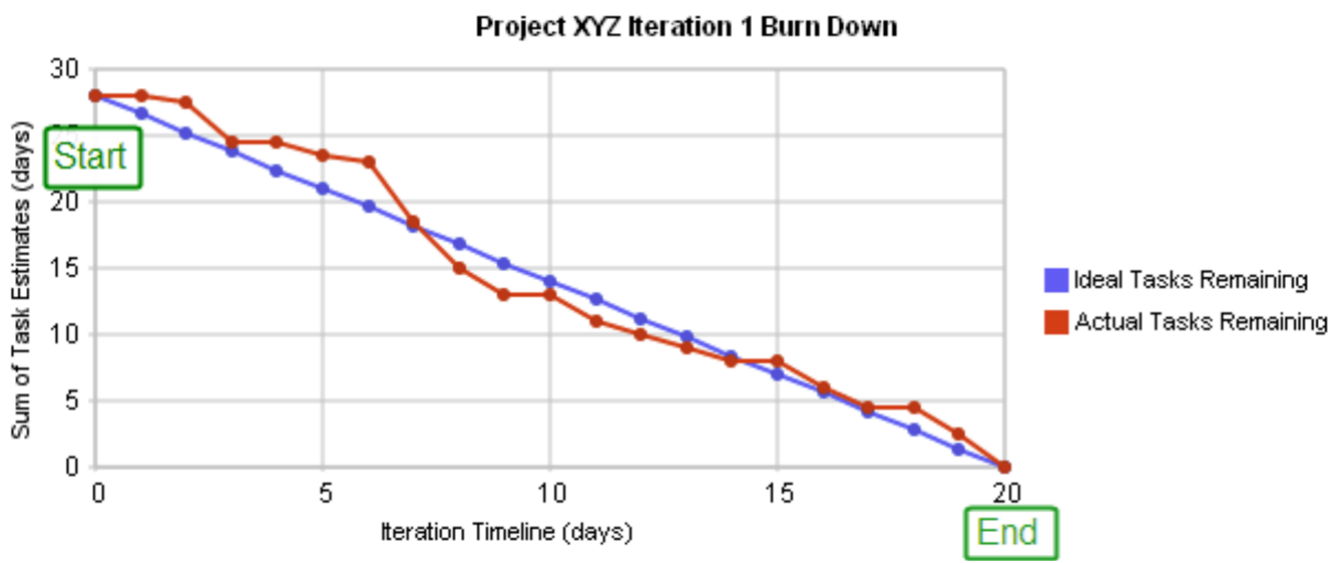
整体日程计划如下：

周期	任务	开始时间	结束时间
Sprint 1	需求开发	Day 1	Day 1
	系统架构设计	Day 2	Day 2
	DevOps 环境搭建	Day 2	Day 4
	用户基础功能设计	Day 3	Day 4
	测试	Day 4	Day 4
	评审	Day 4	Day 4

Sprint 2	需求、工作项确认	Day 5	Day 5
	选课管理基础功能开发	Day 6	Day 8
	系统邮件功能开发	Day 6	Day 6
	自然语言交互接口开发	Day 6	Day 7
	测试	Day 8	Day 8
	评审	Day 8	Day 8
Sprint 3	需求、工作项确认	Day 9	Day 9
	自然语言交互功能完善和开发	Day 9	Day 10
	系统性能和安全优化	Day 9	Day 10
	单元测试和集成测试	Day 11	Day 12
	评审	Day 11	Day 12
After Sprint	用户培训	Day 13	Day 14
	仿真模拟	Day 13	Day 14

燃尽图项目跟踪管理

使用一个公开展示的图表，显示当前 Sprint 中未完成任务数目，或在 Sprint 订单上未完成的订单项的数目。使得冲刺平稳覆盖大部分 Sprint 周期且使项目仍在计划周期内。



质量计划

项目小组在软件开发过程中将按照下述标准执行下列质量保证活动：

过程记录和分析

- 记录工作过程：开发人员需要记录自己的工作过程，包括工作时间、工作内容、工作量、工作效率等。使用时间追踪工具，如 Toggl、Clockify 等，记录工作时间和任务分配，并记录工作中涉及的任务，遇到的问题，解决方案和取得的进展。
- 分析工作效率和质量：开发人员根据 Toggl、Clockify 等工具的记录，分析自己每日的有效工作时间和工作效率高的“流时间”，回顾今日工作中遇到了哪些比较困难的问题，在哪些问题上花费了过多的时间，代码质量如何（在代码评审阶段中出现的问题）。
- 思考挑战和改进点：开发人员根据上一步分析的结果，思考如何更好地安排自己的工作时间，以提高自己的“流时间”或有效工作时间；思考问题困难或占用过多时间的本质原因，例如缺乏知识、技能不足、时间管理等。

编码规范

为保证项目的代码质量和代码规范，开发小组制定如下规范：

- 对于服务端开发而言，开发人员需遵守 Google 制定的 [Go 编程规范](#)；对于客户端开发而言，开发人员需遵守 Google 制定的 [JavaScript 编程规范](#)。
- 开发人员需分别使用 JetBrains 开发的 GoLand 和 WebStorm 集成开发环境进行编程，在编程的过程中，分别使用 [sonarlint](#) 和 [eslint](#) 插件对代码进行静态分析，消除所有 Minor 及以上级别的警告提示。
- 开发人员需使用 [Testify](#) 框架为代码编写单元测试、集成测试，代码测试的细节详见检测性质量管理。
- 开发人员应使用 Git 版本控制系统进行代码版本管理，并遵守开发小组制定的 Git 管理规约，确保代码的可追溯性和可恢复性。
- 在代码发布之前，开发人员应进行代码评审来保证合并代码的质量，详见产品质量评估。

单元测试

单元测试需要对所有设计中承担核心功能（超过 3 个模块可能调用）或对应关键需求（见需求规格说明书）的方法编写详尽的单元测试。代码提交之前，需要进行回归测试并通过所有测试用例，方可入库。本项目的单元测试质量要求如下：

- 测试覆盖率：确保代码覆盖率达到 98% 以上。建议使用 Golang 的 cover 工具检测代码覆盖率，并确保每个方法和分支都得到适当的测试。
- 模拟与隔离：对于承担核心功能或对应关键需求的类进行模拟与隔离，建议使用 GoMock 等框架模拟外部依赖和隔离被测代码，以确保单元测试的独立性和可重复性。
- 快速执行：保证单元测试的快速执行，确保在每次构建过程中都能够迅速完成单元测试，建议任一次单元测试用时不超过 30s，若违反此时限，建议检查、精简测试用例。
- 设计单元测试的测试用例时，需要注意以下条目：
 1. 独立性：每个测试用例应该是相互独立的，不受其他测试用例的影响。避免测试用例之间的依赖关系，以确保每个用例都可以独立运行和验证。
 2. 边界条件：考虑输入和输出的边界条件。针对边界条件设计测试用例，包括最小值、最大值、边界值、边界范围等，以确保系统在边界情况下的正确性。
 3. 分支覆盖：针对代码中的分支和条件语句设计测试用例，以确保每个分支都得到覆盖。包括测试真值和假值的条件，以及多个条件组合的情况。
 4. 边界覆盖：对于有范围的输入参数，设计测试用例以覆盖边界范围的情况。
 5. 错误处理：设计测试用例来测试错误处理和异常情况下的代码路径。验证系统的异常处理逻辑，检查系统是否能够正确处理错误输入、异常情况。

6. 数据驱动测试：采用数据驱动的方法设计测试用例。选择不同的测试数据集，包括正常数据、边界数据、异常数据等，以覆盖不同的情况和情景。
7. 可读性和可维护性：编写清晰、简洁、易于理解的测试用例。使用有意义的命名、注释和断言，以便其他人能够轻松理解测试用例的目的和预期结果。
8. 验证预期结果：每个测试用例都应该有明确的预期结果。使用断言来验证实际结果是否与预期结果一致。
9. 全面性：尽量覆盖不同的代码路径、边界条件和异常情况。考虑不同的输入组合、参数组合和代码执行流程，以确保测试的全面性。
10. 可重复性：确保测试用例是可重复执行的。每次在相同条件下运行相同测试用例时，应该得到相同的结果，不受外部环境或其他因素的影响。

集成测试

系统中每一个模块正式发布之前，相应负责人应该进行详尽的集成测试，确保对不同模块之间的集成进行充分的测试覆盖。测试应包括模块间的接口、数据传递和依赖关系等。本项目的集成测试质量要求如下：

模块通用要求

- 测试范围确定：明确定义集成测试的测试范围，包括需要集成测试的模块、组件或子系统。根据项目需求和架构设计，确定需要进行集成测试的部分。
- 接口定义和验证：对模块之间的接口进行明确定义，并验证接口的正确性和一致性。确保数据传递、方法调用和消息传递等在不同模块之间可以正确地进行。
- 接口兼容性：测试模块之间的接口兼容性。确保不同模块之间的接口能够正确匹配和兼容，以确保系统的整体功能和性能。
- 数据一致性：验证在不同模块之间传递的数据的一致性和正确性。确保数据在传递过程中不会丢失、损坏或篡改，以及正确处理隐私信息加密 / 解密，数据的转换和格式化。
- 依赖管理和隔离：管理模块间的依赖关系，并进行必要的隔离。确保每个模块的测试独立性，排除外部依赖的影响，并使用 Testify 或 GoMock 工具来模拟依赖的行为。
- 错误处理和异常情况：测试模块之间的错误处理和异常情况。使用 Testify 测试框架和验证系统在异常情况下的行为和处理逻辑，确保系统能够正确处理错误输入、异常情况和边界条件。
- 日志和报告：记录测试过程中的日志和报告。包括测试结果、错误信息、异常情况、测试覆盖率等，以便后续分析和改进。
- 测试管理和跟踪：使用 Testify 框架进行测试用例管理和跟踪。确保每个测试用例都能够被追踪和管理，以便及时发现和解决问题。

用户界面模块要求

- 用户界面测试：验证用户界面的可用性、易用性和一致性。
- 测试数据准备：准备各种课程的数据以及多种用户类型（包括教务员，教师，学生等），并确保覆盖各种情况和边界条件。
- 安全性和稳定性：测试系统面对 DDoS 攻击、SQL 注入、XSS 攻击等安全问题的能力。使用安全测试工具，如 OWASP ZAP 等，对系统进行安全性测试，确保系统能够抵御常见的安全攻击和漏洞，保护用户的数据和隐私。
- 并发和并行测试：测试系统的并发和并行处理能力。使用 Apache Bench 工具，模拟多用户同时进行选课的场景，验证系统在多线程、多进程、多用户等并发情况下的正确性和性能，确保系统能够稳定运行和高效处理并发请求。

数据持久化模块要求

- 数据库测试：使用 DB Unit 工具针对数据库操作和数据一致性进行测试，验证数据库的读写操作（包括数据插入、更新、删除和查询）的正确性和性能。

功能、性能、安全性评估

功能、性能和安全性评估是确保系统质量和性能的关键步骤，它可以以用户视角最直观地发现问题、预防缺陷。整体性功能（性能、安全性）评估会在项目收尾阶段进行，预计进行 1-2 次。

- 评估目标：
 - 功能评估：确保系统满足所有功能需求。
 - 性能评估：评估系统的性能，包括响应时间、吞吐量和并发性能。
 - 安全性评估：评估系统的安全性，包括身份验证、授权、数据加密等。
- 评估内容：
 - 功能评估：对照需求规格说明文档和详细设计文档，确认所有的需求均已实现，可以使用 Selenium 等工具模拟与用户的交互细节。
 - 性能评估：对照需求规格说明文档，确认开发团队承诺的性能要求和并发耐受度是否满足要求。
 - 安全性评估：对照需求规格说明文档，识别可能的功能风险、性能风险、安全性风险等内容，测试系统的耐受程度。
- 报告和反馈：
 - 评估报告：在上述评估完成后需要撰写详细的评估报告，包括评估结果、问题列表和优化建议。
 - 反馈会议：开发团队之间需要分享评估结果，讨论问题和解决方案。

制定个人目标和计划

- 设定明确的目标：开发人员需要设定具体、可衡量、可实现的个人目标，例如学习特定技术、提高代码质量或改进时间管理等。
- 制定计划和里程碑：开发人员需要根据自己设定的目标制定可行的计划和里程碑，将目标分解为具体的任务和时间表，设定截止日期以保证自己的工作效率。
- 考虑资源和限制：开发人员需要考虑自己能够获得的资源，个人的限制和其他优先事项，以确保计划的可执行性。

过程改进和学习

- 持续学习：开发人员需要根据计划积极寻找资源并持续学习。参加培训、课程、讲座等，不断学习和积累知识。
- 实践和应用：开发人员需要实践和应用所学知识，尝试新的方法和技术，并记录在尝试这些新的方法和技术后，工作情况发生了哪些变化。
- 反馈和评估：开发人员需要定期回顾自己记录的内容，并通过代码评审等手段获取他人的反馈意见，评估自己当前的工作质量和改进点。
- 调整和优化：开发人员需要根据反馈和评估中得到的改进点调整工作方法和学习计划，寻找优化的机会。
- 持续改进：个人过程改进是一个持续的过程，开发人员需要不断学习、尝试、反思和调整，以不断提高自己的工作效率和质量。

需求确认和验证

对于需求确认和验证部分，开发小组需要确保：

- 详细分析和确认在线选课系统的需求，确保对系统的理解准确并与客户的期望一致。
- 与利益相关者（如教务员、学生、管理员等）密切沟通合作，获取他们的需求和反馈。
- 确保需求规格文档清晰、一致、可追踪，并包含必要的功能、性能和安全性需求。
- 使用需求验证技术，如原型设计、模拟演示或用户故事验证，以确保需求的正确性和可行性。

代码评审

代码评审可以确保代码质量，降低缺陷率，提高系统可维护性。

- 评审要求：代码入库前进行评审，系统整体代码评审覆盖率需高于 98%。所有新功能、缺陷修复和重要代码修改均需进行自底向上的评审，至少每个迭代周期进行一次自顶向下的全体参与的代码评审。
- 评审参与者：
 - 主持人：评审的负责人，负责安排评审会议和跟踪问题进度。通常为项目负责人。
 - 作者：编写代码的开发人员。
 - 评审者：对代码进行评审的团队人员，至少包括一位非作者的评审者。
- 评审内容：
 - 功能完整性：代码是否完整实现了需求。
 - 代码结构：代码是否模块化，遵循设计原则。
 - 命名规范：变量、函数和类的命名是否符合规范。
 - 性能影响：代码对系统性能的影响。
 - 错误处理：是否考虑了异常情况和错误处理。
 - 防御性编程：是否考虑了边界并进行了不可能情况判定。
 - 兼容性：是否能在不同的系统和环境上正常运行。
- 度量和数据要求：
 - 代码覆盖率：评审的代码测试中代码覆盖率需要达到 95% 以上。
 - 静态代码分析工具：在评审时需要再次使用 sonarlint 或 eslint 静态代码分析工具，确保所有 Minor 及以上警告均已消除。
 - 复杂度度量：评估代码复杂度，确保不超过项目对应模块要求的阈值。
 - 注释率：代码中注释的比例不低于 20%。
- 评审会议：
 - 会议时间：评审会议的安排应在代码提交的 48 小时内完成。
 - 评审记录：记录评审会议的讨论、发现的问题和解决方案。
 - 问题跟踪：使用问题跟踪工具追踪评审中发现的问题，确保它们被及时修复。
- 学习和改进：代码作者和评审人的周报中应该体现代码评审的会议内容、结果、参会者的个人感悟等。

软件设计

在设计方面，开发小组努力做到：

- 参考使用标准的设计模式和最佳实践，提高代码的可读性和可重用性。
- 设计系统架构和模块划分，确保系统具有良好的可扩展性和可维护性。
- 将功能模块划分为独立且高内聚的模块，确保模块之间的耦合度尽可能低。
- 考虑系统的安全性和数据保护措施，以防止潜在的安全漏洞。

潜在冲突协调 - 质量保障活动对正常开发活动的影响

冲突问题

为了保证系统能够承受 1000 人左右的并发量，同时满足快速稳定等质量属性，团队成员需要采取诸如评审、测试等质量保障活动来保证软件项目的质量，这些活动可能会打破项目原计划的正常开发活动，导致原本开发活动被推迟甚至被重新设计实施。

解决方案

1. 质量保障活动存在于团队成员的每个 Sprint 计划中，并保证了每个 Sprint 周期在正常情况下足以开展所需的质量保障活动；
2. 团队计划的开发时间是 2 周，而我们计划的 Sprint 为 12 天，故如果 Sprint 开发过程按预期结束，团队可以提前上线系统开展仿真测试和用户培训。

质量监控指标

开发团队拟采用 Yield、A/FR、PQI 和 Review Rate 四个指标监控该系统的质量。

Yield

Yield 指标用以度量每个阶段在消除缺陷方面的效率。Yield 指标的计算公式如下：

$$\text{Phase Yield} = 100 * (\text{某阶段消除的缺陷个数}) / (\text{某阶段注入的缺陷个数} + \text{进入该阶段前遗留的缺陷个数})$$

以下为质量总体目标预测表：

ID	阶段	时间（人时）	Phase Yield	缺陷注入率	注入缺陷数	消除缺陷数	剩余缺陷数
1	需求开发	12	0	0.75	9	0	9
2	需求评审	12	0.67	0	0	6	3
3	概要设计	8	0	0.75	6	0	9
4	概要设计评审	4	0.44	0	0	4	5
5	详细设计	16	0	1	16	0	21
6	详细设计评审	8	0.57	0	0	12	9
7	DevOps 开发	8	0	1	8	0	17

ID	阶段	时间 (人时)	Phase Yield	缺陷注入率	注入缺陷数	消除缺陷数	剩余缺陷数
8	DevOps 评审	4	0.35	0	0	6	11
9	编码	86	0	2	172	0	183
10	代码评审	36	0.72	0	0	132	51
11	测试开发	10	0	0.125	2	0	53
12	单元测试	8	0.30	0	0	16	37
13	集成测试	16	0.54	0	0	20	17

A/FR

A/FR = PSP 质检成本 / PSP 失效成本。在本项目中，PSP 质检成本与失效成本由以下几个部分组成：

质检成本 = 设计评审时间 + 代码评审时间 = $(4 + 8) + 36 = 48$ 人时

失效成本 = 编译时间 + 测试时间 = $0 + (8 + 16) = 24$ 人时

$$A/FR = 48/24 = 2.0$$

- 失效成本中编译时间记为 0，是因为编译时间通常较短，不会对项目的进度和成本产生显著影响。而测试时间较长，是项目中的一个重要环节，对项目的质量和进度有较大的影响。
- A/FR 指标越高，说明质检成本越低，失效成本越高，质量管理效果越好。但过高的 A/FR 指标往往意味着过多的评审和测试活动，可能会影响项目的进度和成本。因此，需要根据项目的实际情况和需求，合理控制 A/FR 指标，确保质量管理活动的有效性和效率。
- 我们计划将此项目的 A/FR 指标控制为 2.0，即质检成本为失效成本的 2.0 倍。这样可以确保项目的质量管理活动得到充分的保障，同时避免过多的评审和测试活动，保证项目的进度和成本。

PQI

PQI 指标用以度量项目的质量水平和质量管理活动的有效性，其值为以下五个过程指标的乘积：

$$\text{设计质量} = \min(\text{设计时间}/\text{编码时间}, 1) = \min(60/86, 1) = 0.70$$

$$\text{设计评审质量} = \min(2 * \text{设计评审时间}/\text{设计时间}, 1) = \min(2 * 12/30, 1) = 0.80$$

$$\text{代码评审质量} = \min(2 * \text{代码评审时间}/\text{编码时间}, 1) = (2 * 36/86, 1) = 0.84$$

$$\text{代码质量} = \min(20/(\text{编译缺陷密度} + 10), 1) = \min(20/(13 + 10), 1) = 0.87$$

$$\text{程序质量} = \min(10/(\text{单元测试缺陷密度} + 5), 1) = \min(10/(6 + 5), 1) = 0.91$$

$$PQI = \text{设计质量} * \text{设计评审质量} * \text{代码评审质量} * \text{代码质量} * \text{程序质量} = 0.70 * 0.80 * 0.84 * 0.87 * 0.91 = 0.37$$

PQI 指标越高，说明项目的质量水平越高，质量管理活动的效果越好。根据我们的估算，我们计划将此项目的 PQI 指标控制为 0.37 左右。

Review Rate

Review Rate 指标是一个用以指导软件工程师开展有效评审的指标。高质量的评审需要软件工程师投入足够的时间进行评审，通常来说 Review Rate 越低，就代表着评审的质量越高，评审越细致。

- 根据前文 WBS 中的估算，我们的代码总行数在 **2400 行左右**。对于本项目，我们计划将代码评审速度控制在约 **200 LOC/时**，代码评审总时间为 **36 人时**。

- 本项目主要包含如下文档：需求规格说明、概要设计文档、详细设计文档、测试文档等。根据开发团队的估算，最终形成的有效的文档页数约为 **60 页**，我们计划将文档评审速度控制在 **3 Page/时**，文档评审总时间为 **24 人时**。

个人计划和计划平衡

开发团队的所有成员在项目周期内均为全职参与者，要求每个成员每天工作 8 小时，分为上午与下午两个 4 小时的时段，考虑到进入流状态需要一定时间，因此认为每天的**有效工作时间为 5 小时**。

为完成预估需要 270 人时的选课系统开发任务，**每个成员可休息 4 个时段合计 2 天**，该休息时段动态调整。由于团队成员之间的工作效率和工作难度不同，因此每个成员的工作量和工作计划也会有所不同。

同时，根据以下因素调整个人日程安排：

- 成员个人原因请假。
- 项目开发流程中的 FS 关系，任务之间是否可并行开发。
- Day 4、8、12 为 Sprint 的评审和回顾日，所有团队成员均参与评审和回顾会议。
- Day 13、14 为用户培训与仿真模拟 / 用户验收测试，所有成员均参与。

以此为基础，我们初步制定了如下的个人日程安排，并且平衡了该计划，以 WBS 编号与成员 / 时段规划，省略覆盖整个开发流程的工作，以 R 指代 Sprint 的评审会议与回顾会议：

日程	Day 1		Day 2		Day 3		Day 4		Day 5				
	9-13	14-18	9-13	14-18	9-13	14-18	9-13	14-18	9-13	14-18			
李薛成	1, 5, 6		10	8, 9	11	13, 20, 36	-	17, 35, R	19, 21, 22	19, 23, 24			
张铭铭			7		12				18, 21, 22	18, 23, 24			
陈皓鑫			10						21, 22	23, 24			
张哲恺			12, 13		14, 15	16	-		18, 29				

日程	Day 6		Day 7		Day 8		Day 9		Day 10		
	9-13	14-18	9-13	14-18	9-13	14-18	9-13	14-18	9-13	14-18	
李薛成	-	18, 19	32, 33	27, 28	13, 35	35, 36, R	18, 19, 30, 31		-	36, 37,38	
张铭铭			25, 26	18, 19, 27			18, 19	34			
陈皓鑫				18, 19, 28							
张哲恺			29	30, 31			18, 19, 30, 31				

日程	Day 11		Day 12		Day 13		Day 14	
	9-13	14-18	9-13	14-18	9-13	14-18	9-13	14-18
李薛成	-	35, 38	35, R	35, 38, 39, 42				
张铭铭								
陈皓鑫								
张哲恺				40, 41				

风险评估

风险识别

团队将项目中可能出现的风险进行了初步的识别，并约定如下：

- 风险类别有：需求风险、技术风险、外部风险、人力风险、质量风险、成本风险。
- 风险优先级有 1、2、3 三种级别，数字越大优先级越高。
- 风险可能性有 1、2、3、4、5 五种级别，数字越大，可能性越大。

风险 ID	风险类别	风险内容	风险条件	风险影响	风险干系人	风险优先级	风险可能性
1	需求风险	项目需求与校方要求不符	需求分析不清晰	需要进行需求变更，重新产出需求规格说明书和产品待办列表，并导致项目延期	产品负责人、开发团队	3	1
2	需求风险	未来学校对选课系统的需求发生变化	教育相关法规和学校政策的变化	需要进行需求变更，可能要重新进行系统设计	产品负责人、开发团队	2	2
3	技术风险	项目技术难度较大，开发团队难以完成开发任务	技术难度大，开发团队技术水平不足	项目延期	开发团队	2	1
4	技术风险	数据库数据丢失或损坏	数据库操作失误	数据丢失，需要恢复数据	开发团队	3	1
5	外部风险	服务器无法正常工作	服务器故障或学校机房断电等	服务器宕机，无法正常使用	开发团队	1	1

风险 ID	风险类别	风险内容	风险条件	风险影响	风险干系人	风险优先级	风险可能性
6	外部风险	数据库数据泄露	未经授权的数据访问或数据库安全漏洞	重要隐私数据泄露，导致严重的安全问题和信任危机	开发团队	3	4
7	人力风险	团队成员无法参与项目开发	团队成员生病或其他不可抗力因素	项目延期	Scrum Master、产品负责人、开发团队	2	1
8	质量风险	用户和 LLM 模型交互时，LLM 实际表现与用户预期不符	LLM 模型性能不佳，训练不足	LLM 无法正常使用，影响用户体验	产品负责人、开发团队	2	5
9	质量风险	选课系统无法支持高并发使用	系统或服务性能不足	选课系统响应缓慢或崩溃，无法正常使用	Scrum Master、产品负责人、开发团队	3	4
10	成本风险	项目成本超出预算	项目成本估算有误	项目延期或需求变更	Scrum Master、产品负责人	3	2

风险应对

风险 ID	应对策略	具体措施	优先级
1	风险解决	重新召开需求会议，进行需求获取、分析、验证等工作，产出需求规格说明书和产品待办列表	3
2	风险缓解	与学校相关部门保持沟通，及时了解学校政策变化	2
3	风险缓解	增加人手或减少功能完整度	2
4	风险转嫁	将数据库管理相关服务交给第三方工作者去完成	1
5	风险转嫁	服务器数据备份，并使用学校提供的服务器	3

风险 ID	应对策略	具体措施	优先级
6	风险转嫁	由学校信息安全部门负责数据库安全	3
7	风险缓解	团队成员之间相互协助，或增加人手，完成项目	2
8	风险转嫁	更换 LLM 服务提供商	2
9	风险缓解	开发团队重构代码，优化系统性能	3
10	风险解决	重新评估项目成本	3

机遇识别和应对

对于开发过程中可能出现的机遇，团队也进行了讨论并给出相关的应对措施，如下：

机遇 ID	机遇内容	机遇影响	应对策略
1	项目开发中发现有更多可复用的组件	项目开发效率提高，减少重复开发	将可复用的组件进行整理，形成组件库，提高开发效率
2	开源社区或校方提供高质量系统框架，具备良好可扩展性，可在此基础上进行开发	节省项目基础框架开发时间，提高开发效率	选用系统框架，减少基础框架开发时间
3	开发效率高于预期，代码编写任务提前结束	提前完成开发任务，提高项目进度	开发团队提前开始下一阶段的工作，并加强后续评审和测试的工作，保证质量
4	GPT-4、Llama 3等大语言模型性能超出预期，能帮助团队完成开发工作	提高项目开发效率，减少人力成本	团队使用大语言模型辅助完成项目开发工作如代码评审、文档编写等